

JS + React Guess-the-Output Question Bank (with Explanations)

Use this for rapid revision. For each question, first predict output, then verify explanation.

1) Event Loop and Task Queue Questions

Q1

```
console.log(1);
setTimeout(() => console.log(2), 0);
Promise.resolve().then(() => console.log(3));
console.log(4);
```

Answer:

- 1
- 4
- 3
- 2

Q2

```
setTimeout(() => console.log("timeout"), 0);
queueMicrotask(() => console.log("microtask"));
console.log("sync");
```

Answer:

- sync
- microtask
- timeout

Q3

```
Promise.resolve().then(() => {
  console.log("p1");
  return Promise.resolve();
}).then(() => console.log("p2"));

console.log("s");
```

Answer:

- s
- p1
- p2

Q4 (Node)

```
setImmediate(() => console.log("immediate"));
setTimeout(() => console.log("timeout"), 0);
process.nextTick(() => console.log("nextTick"));
Promise.resolve().then(() => console.log("promise"));
console.log("sync");
```

Typical Node answer:

- sync
- nextTick
- promise
- timeout/immediate ordering may vary by phase timing context

Interview note:

- Mention that setTimeout vs setImmediate relative order is context dependent.

2) Hoisting and Scope Questions

Q5

```
console.log(a);
var a = 5;
```

Answer:

- undefined

Q6

```
foo();
function foo() {
  console.log("ok");
}
```

Answer:

- ok

Q7

```
bar();
var bar = function () {
  console.log("bar");
};
```

Answer:

- TypeError: bar is not a function

Q8

```
{  
  console.log(x);  
  let x = 1;  
}
```

Answer:

- ReferenceError (TDZ)

3) Closure and Loop Questions

Q9

```
for (var i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 0);  
}
```

Answer:

- 3
- 3
- 3

Q10

```
for (let i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 0);  
}
```

Answer:

- 0
- 1
- 2

Q11

```
function outer() {  
  let x = 1;  
  return function inner() {  
    x += 1;  
    return x;  
  };  
}  
const fn = outer();  
console.log(fn(), fn(), fn());
```

Answer:

- 2 3 4

4) this Binding Questions

Q12

```
const obj = {  
  value: 10,  
  get() {  
    return this.value;  
  }  
};  
  
const g = obj.get;  
console.log(g());  
console.log(obj.get());
```

Answer:

- undefined (or global value in non-strict legacy context)
- 10

Q13

```
function f() {  
  return this.x;  
}  
const o = { x: 7, f };  
console.log(o.f());  
console.log(f.call({ x: 9 }));
```

Answer:

- 7
- 9

Q14

```
const obj = {  
  x: 1,  
  arrow: () => this.x,  
  normal() { return this.x; }  
};  
console.log(obj.arrow());  
console.log(obj.normal());
```

Answer:

- lexical this result (commonly undefined in modules)
- 1

5) Promise and Async/Await Questions

Q15

```
async function a() {  
  return 5;  
}  
a().then(console.log);  
console.log("sync");
```

Answer:

- sync
- 5

Q16

```
async function run() {  
  try {  
    await Promise.reject(new Error("x"));  
    console.log("after");  
  } catch {  
    console.log("caught");  
  }  
}  
run();
```

Answer:

- caught

Q17

```
Promise.resolve()  
  .then(() => {  
    throw new Error("e");  
  })  
  .catch(() => "recovered")  
  .then(v => console.log(v));
```

Answer:

- recovered

Q18

```
Promise.any([  
  Promise.reject("a"),  
  Promise.resolve("b"),  
  Promise.resolve("c")  
]).then(console.log);
```

Answer:

- b

6) Array/Object Questions

Q19

```
console.log([1, 2, 3].map(parseInt));
```

Answer:

- [1, NaN, NaN]

Why:

- parseInt receives (value, index) and index becomes radix.

Q20

```
const a = { x: 1 };  
const b = a;  
b.x = 5;  
console.log(a.x);
```

Answer:

- 5

Q21

```
const x = [1, 2];  
const y = [...x];  
y.push(3);  
console.log(x.length, y.length);
```

Answer:

- 2 3

Q22

```
console.log(typeof null);  
console.log(typeof []);  
console.log(Array.isArray([]));
```

Answer:

- object
- object
- true

7) React-Specific Output/Behavior Questions

Q23: state batching

```
function Comp() {  
  const [count, setCount] = React.useState(0);  
  
  const click = () => {  
    setCount(count + 1);  
    setCount(count + 1);  
  };  
  
  return <button onClick={click}>{count}</button>;  
}
```

Question:

- After one click, count becomes?

Answer:

- 1 (same stale value used twice)

Fix:

```
setCount(c => c + 1);  
setCount(c => c + 1);
```

Then result becomes 2.

Q24: useEffect cleanup timing

```
useEffect(() => {  
  console.log("effect");  
  return () => console.log("cleanup");  
}, [value]);
```

Answer:

- cleanup runs before next effect re-run and on unmount.

Q25: key behavior

Question:

- Why can using index as key cause wrong UI state in reordered lists?

Answer:

- Component identity shifts, so state may attach to wrong item after reorder.

8) Bonus Tricky Questions (Interview Favorites)

Q26

```
console.log(0.1 + 0.2 === 0.3);
```

Answer:

- false

Q27

```
console.log(NaN === NaN);  
console.log(Number.isNaN(NaN));
```

Answer:

- false
- true

Q28

```
console.log("5" - 2, "5" + 2);
```

Answer:

- 3 "52"

Q29

```
const p = Promise.resolve(10);  
const q = p.then(v => v * 2);  
console.log(p === q);
```

Answer:

- false

Q30

```
let x = 1;  
function test() {  
  console.log(x);  
  let x = 2;  
}  
test();
```

Answer:

- ReferenceError (TDZ)

9) How to Use This File Tonight

1. Attempt all 30 without looking.
2. Mark mistakes by topic (event loop, closure, this, promises, React).
3. Re-attempt only wrong ones after 30 minutes.
4. Explain each answer verbally as if teaching interviewer.